Network Influence Stuff

Antoni Zyla (u5527350) - Supervised by Nathan Griffiths

Procrastinated forever

Contents

1 Abstract **3**

2 Introduction **4**

3 Literature Review **5**
3.1 Models of Influence Propogation 5
3.2 Working with large networks 6
3.2.1 Naive Approaches 6
3.2.2 Smarter Sampling Methods 7
3.2.3 Scale Free (or not) Networks 7
3.3 Methods for finding seeds 8
3.4 CoFIM - Community Based Methods 10
3.4.1 Community Detection Algorithms 10
3.5 Working with multiple metrics 11
3.6 Evaluating network topology 11
3.6.1 Approximations of metrics 12

4 Problem Formulation **13**

5 Proposed Algorithms **14**
5.1 Hybrid Algorithms 14
5.2 Community Aware Algorithms 16
5.2.1 Saturating or Connecting Communities 17
5.2.2 Exploiting Community Detection Algorithms 17
5.3 Limited Context Algorithms 18

6 Experimental Method and Setup **19**
6.1 Datasets 19
6.2 Sampling Methods 19
6.3 Accounting for differences in network sampling 19
6.4 Evaluation Methods 21
6.5 Baseline Behaviours 21
6.6 Statistical Analysis 21

6.6.1 The data is a curve 23
6.6.2 Generalising to networks in General 23
6.7 A note on perofrmance 23
6.8 Tools and Libraries 23

7 Results and Analysis **25**
7.1 Hybrid Algorithms 25
7.1.1 Parameters 25
7.1.2 ICC Analysis 25
7.1.3 LTM Analysis 29
7.1.4 DCC Analysis 31
7.1.5 Subsection of Deg/mnd across parameters 34
7.1.6 Methods that Did not work 34

8 Limitations **36**

9 Conclusion **37**

1 Abstract

Approx 200 - 300 words as a very high level overview.

2 Introduction

Approx 600 - 900 words as a broad overview of the problem, challenged, direction investigated and results.

If we were a marketer with access to a population of people and some information about the relationships between them, we may choose a subset of this population who would most effectively market our product. We can consider a population of people in a graph as nodes with relationships between them as edges. Using a model of conceptual contagion we can choose a subset of nodes to act as seeds from which ideas propogate, they may have a direct chance of propogating to their neighbours or another non direct effect.

This selection problem is NP-Hard

3 Literature Review

I have considered a wide range of sources so far including research papers, doctoral theses, open source datasets, articles and open source code. Each piece of information is cited or linked as a footnote where appropriate.

3.1 Models of Influence Propogation

There are three main models through which influence is typically spread across networks.

1. Independent Cascade Model (ICC) - Each node can take a state out of Active, Inactive and uninfected. Each active node has a probability p to infect each of it's uninfected neighbors. Once a node has been infected this process starts again to all of it's neighbors, the node is in the active state once it is infected [10]. Once a node has attempted to infect all of it's neighbors it becomes an inactive node. This is the simplest of the models but also mimics real life well, weather it's the spread of a new software technology or a new restaurant becoming more popular.

2. Decreasing Cascade Model (DCC) - A modification of the ICC model where every attempt of a node to be influenced decreases the probability of it being influenced or infected [11]. This could be analogous to a common experience, overreaching of advertisements or correspondence, such as marketing emails which have the opposite to intended effect due to annoyance.

3. Linear Threshold Model (LTM) - Every node v is given a probability of being turned p which is uniformly distributed across the interval $[0, 1]$. When the proportion of node v's neighbors which are infected is greater than or equal to p, v itself becomes infected [10]. This models well another common experience, a group reaching a common view in regards to something, whether it's the adoption of a new game, or even a software tool that becomes the 'industry standard' this principle is at play.

Various papers have described modifications of each of these models in a multitude of ways, multiple concepts can be spread wherein they can inhibit the spread of eachother by allowing a node to only adopt a single concept [1, 6]. Vaccination or heard immunity can be modelled in this manner [1].

There are other models which work in different manners, borrowing from biological models such as SIR for modelling diseases more closely [12]. These are somewhat able to be modelled using the cascade models however full parity is not possible for example going from infected back to uninfected is not a behavior that is considered with the cascades and would possibly have large effects on the effectiveness of certain seed selection strategies. It is possible to take an agent based approach to influence giving each agent certain characteristics

Social networks, communication systems, power grids, logistics systems are constantly evolving systems. Nodes are constantly added or removed, connections between nodes are frequenrly added, removed or rerouted, working on a snapshot of an ever evolving network may not be the most effective method for real world problem solving. Dynamic models of influence are models where paths between nodes can be added or removed as part of the process [14], this can for example be used to model the evolution of social networks, and how influential nodes may become more or less influential based on different factors or topology, rather than studying networks which are at a fixed point in their evolution. Influence can be studied over these evolutions, however this is an additional very large jump in complexity and scope which would be out of scope for this project.

Another way in which this field has evolved is the study of adversarial contagions. Analogous to the spread of a disease against a vaccination, two concepts/contagion spreading across the network at the same time, each node can only be infected with one and may or may not be turned from one to the other. (SOURCE)

3.2 Working with large networks

Some of the networks I will be studying are made up of tens of thousands of nodes with millions, perfoming non trivial analysis on networks of this size is almost impossible with reasonable compuitng resources.

Sampling networks is a way of preserving the topological features and general properties of a network. This allows us to create strategies by quickly testing on small networks to what works and what doesn't and then due to being a representative sample we know that they will generalise well to larger networks.

3.2.1 Naive Approaches

A simple natural approach could be to take a node at random from the graph, call it v, and then perform a breadth first search in layers across the graph. When a high degree node is encountered then each of it's neighbours would also be added, this will inflate the average degree of the network by tending to sample a subset with a higher mean degree [16].

Working outwards from a start node also does not preserve the distribution of shortest path distances between nodes or clustering coefficients well. For node located close to the root these values will remain quite accurate but as we move further and further out a node is less likely to appear before we have no more nodes needed leading to a lower shorter mean path length and a higher typical clustering coefficient over other methods [4].

Some papers describe BFS as adding the entire layer from which it explores, leading to oversized samples [2]. For example taking a BFS sample of 1000 nodes in the Enron dataset it can easily produce a subgraph of over 12 thousand nodes which is in no way comparable to another method's 1000 node sample. This

is not useful when comparing like for like network sizes and their respective topological features such as diameter which are closely dependent on overall network size.

Snowball sampling is a very simple variation on breadth first search, once the addition of the final layer would take the total number of sampled nodes over the threshold, nodes are simply added one at a time until the exact number of nodes is reached [2].

A random walk is another simple strategy for exploring a graph. Working on the basis of a standard random walk, starting from a node v there is a $\frac{1}{deg(v)}$ probability of choosing the next node to be u where u is one of the neighbours of v. intuitively RW will suffer from the same problems as BFS with finding higher degree nodes much more often than lower degrees [4], as a higher degree node has more connections that lead to it.

3.2.2 Smarter Sampling Methods

Utilising modified random walks is a standard method of achieving accurate and unbiased samples of large networks such as facebook [4].

Metropolis Hastings Random Walks and Random Walks with Delayed Acceptance (MHRW and MHDA) are two common methods used [4, 5]

1. Metrpolis Hastings Random Walk (MHRW) - Sample a random uniform probability in the range 0 to 1 and call it p. If for a given node $p \leq$ min

2. Metropolis Hastings Random Walk with Delayed Acceptance (MHDA). This is a further modification of MHRW where nodes with a high degree are less likely to be chosen. Overall producing a more representative sample of the network than MHRW with little extra performance cost.

3. Albatross Sampling (AS) - A random walk based sample with a probability of jumping to a random node within the network, this should allow for better capture of networks with disconnected or densely clustered components or communities on a very large scale.

3.2.3 Scale Free (or not) Networks

A scale free network is one where the distribution of individual node degrees could be modelled using an exponential function. In this case the probability of a node having a degree k is modelled by the below equation.

$$P(k) \sim k^{-\gamma} \tag{1}$$

This means that there exists a few nodes with a very high number of connections with the majority of nodes having a low number of connections. Some examples of scale free networks are typical human social networks or even wikipedia articles. A simple way to visualise this is by looking at your facebook or instagram account, you and the majority of people may on average have maybe

20-50 friends or connections, checking the numbers for any actor or influential figure may yield values in the millions.

If a large proportion of the network is able to quickly reached by these very high degree nodes then this informs that the value of a degree based heuristic may prove to be more valuable than any other heuristic.

In contrast networks that do not exhibit this behaviour are often not as easily spread through using basic heuristics. Exploiting community structures may be more worthwhile or taking into account the nodes around a particular node.

3.3 Methods for finding seeds

There are a few methods which don't rely on a heuristic to determine the seed nodes.

1. Brute Force - for a graph made up of n nodes there are n choose k possible sets of starting seeds. Running a cascade for each of these would be incredibly inefficinet and impractical by being on the order of n factorial. I have not been able to find reference of this being used to compare with other methods likely due to the required processing power.

2. Greedy - Starting with an empty seed set choose the node in the subgraph of the network bar our seeds which will produce the largest influence, add this node to our set of seeds and continue to add nodes until we have a set of our desired size. This achieves a $1 - 1/e$ approximation ratio to the optimal solution due to (TODO)

3. Cost Effective Lazy Forward (CELF)

Other factors which can help us guage how influential a node may be with no global knowledge of the graph required are:

1. Degree (Deg). In the ICC and DCC models a node has some chance p of spreading a concept to it's neighbours it is clear that on average it would spread to an expected $p \deg(v)$ nodes. Likewise on threshold based models a high degree node being chosen will increase the proportion of infected nodes for it's very high number of neighbours triggering a likely large cascade.

2. Degree Discount (DD). It's known that high degree nodes tend to cluster together, choosing multiple tightly coupled high degree nodes is redundant and not as effective as it could be. Removing any neighbours of a selected node and then using degree is an effective way of minising the overlap of expected influence between chosen seed nodes. Requirign a factor of Δ more computation power than raw degree heuristic it is a performant option. There is a limitation with regards to networks which have a few very high degree nodes, by choosing just a few of these nodes there are

often minimal nodes left to choose from and an appropriate sample is not able to obtained.

3. Mean Neighbour Degree (MND). Thinking about how the cascade will propogate, choosing nodes who's neighbours are very high degree nodes will result in gaining access to multiple influential nodes (those with high degree) from a single seed. Compared to raw degree this requries a factor of Δ^2 more computation. For most graphs this is still a very performant option as those very high degree nodes are very rare.

Those above are simple to work out with no global knowledge of the graph.

1. Local Clustering Coefficient (LCC). Consider a node where each of it's neighbours is connected to every other neighbour, this would be a clique and a very highly clustered node. We are measuring how close a node is to being in a clique from it's neighbours by taking the proportion of neighbours connected to other neigbours from the total possible number. Computing this is relatively fast being just a factor of (TODO) more than a degree based metric. Performance is dependant upon network topology. On a very simple level a highly clustered node will perform well at spreading concepts, each of it's nieghbours will also be highly connected and help spread the concepts further. There are some cases where it may not be of use, if in a graph there exists a small almost unconnected clique of nodes of size greater than our seed size then choosing them all will not produce a good spread in the rest of the graph.

$$C(v) = \frac{2T(v)}{\deg(v)(\deg(v) - 1)} \tag{2}$$

(TODO)

2. Betweenness Centrality (BC). The proportion of shortest paths from any two nodes in the network which pass through the point. Take a graph of two communities with a single edge between them, the two nodes on either side of the edge would have a very high betweeneess as they are vital for the connection of the two communities. Most topologies will not exhibit this extreme a distribution in betweeness however it is still a useful metric for influence. As we need to compute the shortest path between each pair of nodes there is a running time on the order of $O(nm)$ (we can cache TODO). Localised approximations for betweneess have been show to be highly correlated with betweneess but only in certain network topologies.

$$B(v) = \sum_{s,t \in V, s \neq v \neq v} \frac{|\sigma(s,t|v)|}{|\sigma(s,t)|} \tag{3}$$

where $\sigma(s,t)$ denotes the set of shortest paths from node s to t, with the conditional on v is shortest paths which include v. h-betweeneess is only taking into account shortest paths of length less than or equal to h, as

h approaches the diameter of the network it becomes a slower but more accurate approximation.

$$B_h(v) = \sum_{s,t \in V, s \neq v \neq t} \frac{\{p | p \in \sigma(s,t|v), |p| \leq h\}}{\{p | p \in \sigma(s,t), |p| \leq h\}} \tag{4}$$

3.4 CoFIM - Community Based Methods

Talk about community based method - Requires am already partitioned network.

HOW DO YOU DEAL WITH SIZING COMMUNITIES IN NETWORKS THAT DONT HAVE A VERY STRONG COMMUNITY STRUCTURE, AT WHAT POINT DO COMMUNITIES BECOME WORTHLESS DUE TO NON COMMUNITY STRUCTURE

How do we define how strongly a network is split up into distinct communities? For a network partitioned into c communities the modularity is defined as

$$Q = \frac{1}{(2m)} \sum_{vw} \left[A_{vw} - \frac{deg_v deg_w}{(2m)} \right] \delta(c_v, c_w) = \sum_{i=1}^{c} (e_{ii} - a_i^2) \tag{5}$$

where e_{ij} is the proportion of edges connecting communities i and j, a_i is the fraction of ends of edges which end in community i. What is the null model in this scenario (TODO) - assuming a random graph where each vertex has a random probability of being connected to any other vertex. What we are finding via Q is how much different the proportion of edges between communities vs inside communities is in our network to the expectation of a random graph that has the same number of nodes and edges as our graph.

If a network displays a strong community structure then that implies there exist subsets where it is hard for ideas to propogate between them. If you were to choose nodes in just a single community then you might be able to saturate a community with a concept, however this may not provide an optimal global spread of the concept in the network.

In certain scenarios this behaviour may be a good thing, if a community structure was distinct in real world characteritics, maybe consumer spending power or education levels then to propogate ideas would require different approaches and we could model this in a more fine tuned manner.

3.4.1 Community Detection Algorithms

Talk about top down and bottom up methods - greedy modularity and girvan newman greedy - talk about effects of chosoing a different algorithm for ranking the removal/combination of edges.

Introduced in 2006 by MEJ Newman (SOURCE TODO) it was found that there exists an $O(n^2 + nm)$ algorithm for community detection in networks based on the notion of maximizing modularity using a greedy method. Starting

from a spectral method (vectors and matrices???) to very roughly partition the network into a set of communities. From this find vertices which will have the greatest gain in modularity for the whole graph, continuing to change the community each vertex belongs to until you cannot increase the modularity of the graph further.

Girvan-Newman algorithm introduced in 2002 works on the basis of identifying edges which link communities. In the same way that a node can have a betweenness coefficient so can an edge, finding the edges with the highest betweenness (or being critical choke points for connecting sets of nodes). To find communities we can iteratively remove edges from the network until we have found the number of distinct communities

Extendded into Clauset-Newman-Moore greedy algoirtmg for modularity maximisation we work on the basis that each node starts as it's own community. A pair of communtiies are identified which would produce the greatest gain in modularity for the overall network by combining them together. A benefit to this approach is that the algorithm will terminate when there is no more possible gain to modularity and so there is an inherent restriction on the number of communtiies.

There exists another method to find such communities, take the edge betweeness of each edge, find the highest edge betweeness and cut that edge, continue this until you have reached your desired number of communtiies. An inherent weakness of this approach is that the number of communities is difficult to determine, the distribituion of node degrees is hard to discern without computation and it varies wildly between iterations of increasing community numbers.

Depending upon network structure finding the number of communities which would be useful for working with influence models is a difficult problem.

3.5 Working with multiple metrics

TOPSIS - Seems to be a ranking of nodes based on placing each in a vector space based on some metrics, determining "optimal" for each metric and then choosing nodes that have the lowest distance from that optimal point in the space.

Is this good, bad? Time complexity, memory cost, size of constant factors?

Can it be easily approximated with just combining naive heuristics in a single factor. Compare to hybrid influence scores like s(v) = deg(v)*1/lcc(v)

3.6 Evaluating network topology

As a network is a graph made up of nodes and edges it's obvious to consider the number of nodes and edges or how these edges are distributed between the nodes. There are many metrics we can use to gain an insight into the type of network topology we are working with.

1. Diameter - The length of the longest shortest path from any two nodes in the network. Example uses are for example if a graph has a diameter of two then it could be a star graph or if it has a diameter equal to the number of nodes then it must be a ring.

$$D(G) = \max_{v,u \in V} \{|shortestpath(u,v)|\} \tag{6}$$

2. Clustering Coefficient - The mean clustering coefficeint of each node in the network.

$$C = \frac{1}{n} \sum_{v \in V} \frac{2T(v)}{deg(v)(deg(v)-1)} \tag{7}$$

   where $T(v)$ denotes (TODO). This measures how closely clustered nodes are, it's a good measure of community structure with a high clustering coefficinet indicating that nodes can be split into distinct communtiies easily with a low coefficient indicating there is not much community structure.

3. Centralization - a measure in the range zero (0) to one (1) which determines how unequal the distribtion of node degrees is with a star newtork of the same size. A value of one indicates that the network is a star whereas a value of zero indicates that the each node has the exact same degree.

$$Cent(G) = \frac{\max_{v \in V}(\deg(v))}{n} - \frac{mean(\deg_{v \in V}(v))}{n-1}$$

4. Mean degree - A very simple measure of the average degree of each node. High mean degree's show that a network is very dense and likewise low values indicate a sparse network. Small numbers of very high degree nodes do need to be considered as they will skew the average degree higher.

3.6.1 Approximations of metrics

Randomised approximation for betweenness centrality - How good is it. how much quicker is it. Is it worthwhile to use. Is betweeness a worthwhile measure

Discouss the approxmations for local clustering - their runtimes, comparing to worst case of n cubed for high degree ndoes, how much worse do these perofrm, is it worth doing, is n cubed too slow to be useufl if more exact??

4 Problem Formulation

- A network is defined as a graph $G = (V, E)$ where $|V| = n$ and $|E| = m$
- $N(v) = \{(v, u)|u \in V\}$. The first jump neighborhood of v
- $\deg(v) = |N(v)|$
- $N_2(v) = \cup_{u \in N(v)} N(u)$ The two jump neighborhood of v
- Independent cascade model - $S = \{active, inactive, used\}$, $p \sim U[0, 1]$, $t \in [0, 1]$, A node can change from inactive to active to used only. An active node has a p chance of infecting each of it's neighbors
- Decreasing Cascade Model
- Linear Threshold Model
- $\sigma(s, t)$ is the set of shortest paths between s and t.
- $\sigma(s, t|v)$ is the set of shortest paths between s and t which contain the node v.

maximal_influential_set(Network, k) = return the subset of the network which will produce the greatest propogation of ideas if seeded from there.

5 Proposed Algorithms

These are the ideas and directions I have explored over the time of researching this topic and am hoping will produce a measurable difference over known algorithms.

5.1 Hybrid Algorithms

From experimental testing there are clear differences in how certain methods for finding seeds perform based on the number of seeds required.

For example in the Enron dataset, a degree based approach works very well at low seed sizes however doesn't scale well with increased seed sizes. Utilising local clustering coefficient to select nodes produces an opposite pattern, at very low seed counts it performs no better than choosing a set of random nodes. Increasing the number of seeds scales very will with further seeds yielding at least a linear relationship with the final influence spread.

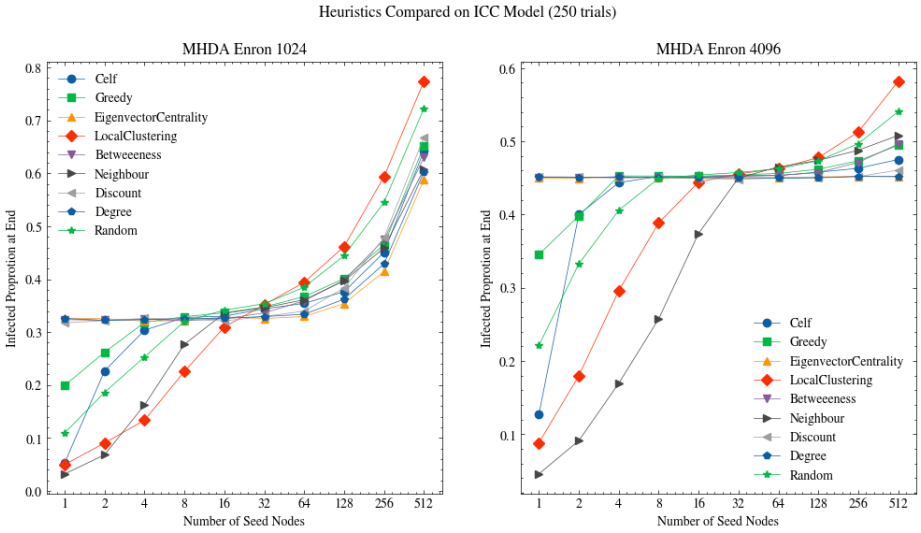


Figure 1: Heuristics Compared on an example network

The general idea this scenario is to utilise a heuristic which performs very well at low seed counts and combine it with a heuristic which performs well at higher seed counts for a significant proportion of our seeds. Based on this logic it may also be a good idea to select two or more heuristics which individually target or infect different parts of our network, where the intersection of the final spread from each individual heuristic would be as small as possible, thus maximising the final influence spread in the network more efficeintly than adding more seeds to the same approach.

Figure 1 shows the relative performance of each of the heuristics ran on a sample of the same network 250 times against a set of seed sizes expoenntially

increasing in powers of two (2) from 1 to 512. Based on this we can see that as expected local clustering coefficient performance incredibly poorly on low seed sizes but is actually the best at high seed sizes on both smaller and larger sampled networks.

1. Degree (Naive, Discount, Double Discount) and Local Clustering coefficient

2. Degree (Naive, Discount, Double Discount) and Betweenness Centrality

Why am I picking the above? - Great question (TODO). Both degree and degree discount based approaches are incredibly efficient at working with low numbers of seed nodes. Based simply on the fact that the expectation of the number of infected nodes is roughly p times the degree of the node. Discount approaches work by attempting to prevent clustering of high degree nodes being selected by removing the neighbours (or also 2 away community) of selected nodes from being selected in our seed sets.

There is a minimal overlap between the nodes chosen by clustering and betweeneess and that of degree (EXPLAIN WHY TODO).

On a large enough network with a sufficiently small sample size and a power law distribution of node degrees there should not be a measurable difference in the performance of degree discount or degree approaches. We however are working with small to medium sized networks, as both a point of comparison with the standard in most papers but also to make computations take a reasonable amount of time.

Choosing nodes based on a 'score' that's made up of a combination of diffferent factors should allow for choosing nodes that will be a good overall node. If choosing some quota of nodes from one method and the remaining nodes from another the chosen set may be good individually but poor overall.

The following are the ideas and reasoning behind them, which produced interesting results across at least one of the network topologies and methods.

1. Custom 1: $S(k) = \deg(\log(k)) \cup bet(k - \log(k))$. As with (5) there is a similar behaviour for betweenness where the final spread from a set of nodes of each method has a small intersection relative to the influenced area.

2. Custom 2: $S(k) = \deg(\log(k)) \cup lcc(k - \log(k))$. Choosing a small proportion with degree and the remaining nodes with clustering coefificient is a nice naive way of choosing these nodes. From exploratory testing there seems to be minimal overalp between the two methods in the finalised spread over a network.

3. Custom 3 $s(v) = \deg(v) \times (1 - lcc(v))$. This will find nodes that are a combination of high degree and low clustering coefficient at the same time. Nodes with a low clustering coefficinet are relatively disconnected

to the clique that would be formed by their neighbours, this would be an excellent way to find nodes which may have the greatest reach through the network in terms of distance and therefore distribution rather than immediate raw node counts.

4. Custom 4 $s(v) = \deg(v) \times \frac{1}{lcc(v)}$. Going to the reciprocal of clustering coefficient will weight the choices more towards the lower clustering nodes than that of degree as in (1).

5. Custom 5 $s(v) = \deg(v) \times \frac{1}{mnd(v)}$. Choosing the nodes with the highest ratio of degree to mean neighbour degrees is quite easy to see quite similar to choosing a low clustering coefficient node because each of the nodes it's connected to will have as low an average degree as possible.

6. Custom 6 $s(v) = \deg(v) \times (1 - \frac{1}{max_{u \in V}(mnd(u))} mnd(v))$. In a similar vein to the relationship between (1) and (2) choose this to weight further towards the degree of a node rather than mean neighbour degree.

7. Custom 7: $s(v) = \deg(v) \times (1 - lcc(v)) \times mnd(v)$. The same logic as with taking high degree low clsutering nodes but weighting for a higher mean neighbour degree.

8. Custom 8: $s(v) = \deg(v) \times bet(v)$

Note that $s(v)$ denotes a score for each node, to choose k nodes simply choose the k nodes with the highest score. $S(k)$ denotes the set of k nodes made up of different sets.

Testing will be done using MHDA samples of each of the four networks with one thousand (1000) nodes, this is lower than the smallest size of our datasets. it will also provide a good comparison to most papers which use networks of around this size. At this scale performance of algorithms such as betweeneesss centrality is more than acceptable. Using larger sizes would require dedicating significant resources to the problem, using approximations for some metrics or using completely different methods that do not rely on these metrics. The naive greedy algorithm would not be feasibly to run on larger networks.

5.2 Community Aware Algorithms

A network having a community substructure is a natural way of thinking about the internal topology of a social network. For example may have a group of friends, other people you may loosely know will have their own group of friends, finding the subset of people who make up the connection between these sets of friends intuitively will have some kind of significance in terms of spreading of ideas between the two groups.

5.2.1 Saturating or Connecting Communities

Depending upon the topology of our network the once a network is partitioned into it's constituent communities to achieve a given final spread and distribution of spread, the approach to take to achieve these goals may vary.

If there is a very prominent community then it may be enough to simply saturate that single community by targeting selection of seed nodes from only the community. This wouldn't achieve a good spread of influence globally across the network but may be the most effective method of achieving the required goals. On the other hand a network without a well defined or strong community substructure may not benefit very much from this approach, finding nodes which in turn are both good nodes within their communities but also are well connected to the other communities.

(TODO FIX THIS IS AWFUL) To measure how strongly a network contains a community structure is modularity, this is a value $Q \in [0, 1]$ which represents the proportion of edges that fall within a community compared to the expected number of edges which would fall inside a community in a random graph with the same number of edges.

Intelligently exploiting these properties of the network and it's subdivided communities may allow for more effectively maximising the spread. TODO TALK MORE ABOUT THIS. HOW DO I DETERMINE IF IT IS WORTH SATURATING A COMMUNITY RATHER THAN SPREADING SEED NODES BETWEEN COMMUNITIES

5.2.2 Exploiting Community Detection Algorithms

As described in section (TODO) there are two (2) main methods to partition the vertices of a network into distinct subsets, removing edges until there are disjoint connected components or by starting with each node in it's own community and combining them by finding the edge with the highest modularity.

When a large community splits down and it's nodes are distributed amongst the other communities, choosing nodes to be split amongst communities equally may cause a lower overall spread than having fewer larger communities.

Investigating this is a good idea as depending upon the network we may not actually get a better result by splitting into further networks (TODO).

When communities are combined into a new community (particularly in high modularity metrics) the edges which are between them can indicate that they are some of the few (very) edges which are connecting individual communities. Using the endpoints of those edges that are combined would likely act as a proxy for nodes which when influenced can spread into multiple communities.

```
function determine_edge_endings(G, k)
merged_edges = []
```

```
for ... (modularity maximisation loop)
merge_edges = [(x, y) for x in U for y in V is (u, v) in G]
merged_edges.append(merge_edges)
endpoints = []
for e in merged_edges:
endpoints.append(e[0])
endpoints.append(e[1])
return endpoints[:k]
```

5.3 Limited Context Algorithms

Consider a network analagous to a network graph held by google, facebook with billions of users, hundreds of connections between nodes. Performing any kind of complex algorithm within such a large network whilst not asympotically slow would be impractical in regards with speed and scale.

Selecting seeds using information obtained locally or without knowledge of the entire network, would be a useful method for working with such large networks. Metrics such as mean neighbour degree may become more valuable, inherently biased and not representative sampling methods may be reframed as a positive, a random walk containing a node itself is an indication that a node is reachable and demonstrates some baseline of influential behaviour.

NEED TO TALK MUCH MUCH MORE ABOUT THIS - NEED TO ACTUALLY IMPLEMENT IN CODE

6 Experimental Method and Setup

This section describes the testing methodology used to conduct experiments, networks used and their properties, and any statistical methods. This is to ensure that each result described can be reproduced with the methods and reasoning clear and simple to understand.

6.1 Datasets

I have used the following real world datasets in my testing. As shown in the table they range in sizes from just over fifteen hundred (1500) nodes to just over eighty thousand (80000) with between seventeen thousand edges (17215) up to over one million and three hundred thousand (1342310).

Name	Nodes	Edges	Mean Deg.	Cluster Coef.	Diam.	Centralization
Enron	36692	183831	10.020	0.497	13	0.00725
P2P	10876	39994	7.355	0.006	10	0.001899
Airports	1574	17215	21.874	0.504	8	0.004334
Twitter	81306	1342310	33.019	0.565	7	0.002114

Figure 2: Real world networks metrics

There is more to these networks such as clustering coefficient, diameter and centralization, the provided datasets show a large range of values for each of these. Note that even though some metrics such as clustering coefficient are possible to be in the range of zero (0) to one (1) it is incredibly impractical to get anywhere close to either of the extreme points.

6.2 Sampling Methods

1. BFS (Breadth First Search). Have a very significant tendancy to lead towards high degree nodes leading to globally non representative samples. We won't use BFS for general sampling however it does have uses for algorithms which may only required a localised view of the network from a certain point.

2. MHDA (Metropolis Hastings Random Walk with Delayed Acceptance). As described in the literature review this is a performant and representative sampling method. We will use this for all samples when comparing the performance of heursitics as it generates represnetative samples as well as being commonly used for this purpose in various papers (TODO CITE).

6.3 Accounting for differences in network sampling

Performing analysis on a single (non deterministic) sample of a network will not create reproducible results. This is due to not producing a representative

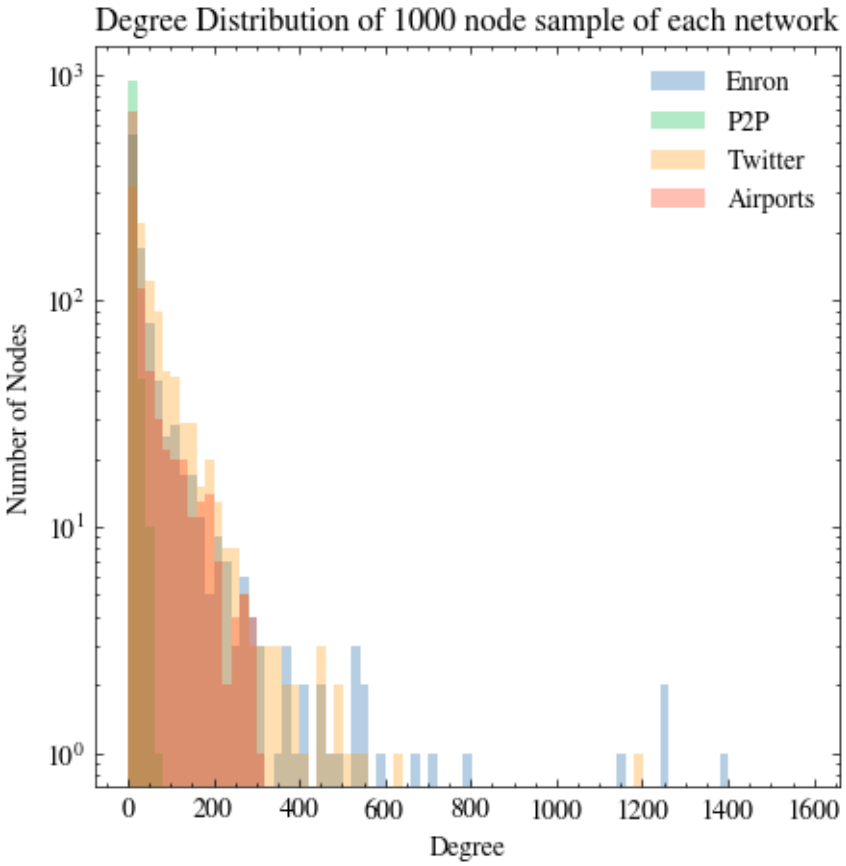


Figure 3: Comparison of Degree Distributions of Networks

sample of the network by each sample having variances in topology, density, connectivity and therefore how well certain heuristics may work.

As most heuristics are deterministic in which seeds they will return for a given network and required size, running the same method on the same network is useful for finding out the noise from the diffusion model rather than variances between different methods.

To perform any statistical analysis of say k runs I will sample the network k times and then each method will be run on each of these networks. This guarantees that the network is covered by each of the samples and will allow us to generalise to the whole network.

6.4 Evaluation Methods

Measuring Final Spread, Rate of Spread, Variability due to changes in parameters, Runtime. All metrics will have a mean, standard dev computed into a tabular format to allow for very quick evaluation of statistical significance.

6.5 Baseline Behaviours

Degree is a very good indicator of influence spread peformance. Single and double degree discount are also excellent.

CELF performs worse than greedy in majority of circumstances, whilst being over 100 times faster.

TOOD talk about why the behaviour shown in the above figures is - highlight the randomness between run to run, talk about cumulative effect of choosing algorithms for a specific network being highly flawed by selected a super high performing heuristic+sample combination on one net that performs very poorly on another.

6.6 Statistical Analysis

There are a few ways to compare our results, it is important that the analysis is rigorous and shows clear statistical significance.

We are comparing the performance of a method across networks, we need to take into account usage of samples within those networks and random variability between runs of each algorithm on each sample. The number of unique samples is very important to have a number high enough. If we had 20 runs per sample with 5 samples of a network it might seem that we have 100 data points to compare. This is not the case, we know that network topology is slightly different between samples and network topology has a large effect on the results of a heuristics performance, each of the sets of 20 runs are correlated with eachother and so cannot be treated as indepeadnt trials. Due to this I will use a linear mixed effects model, for each run of a heuristic I will

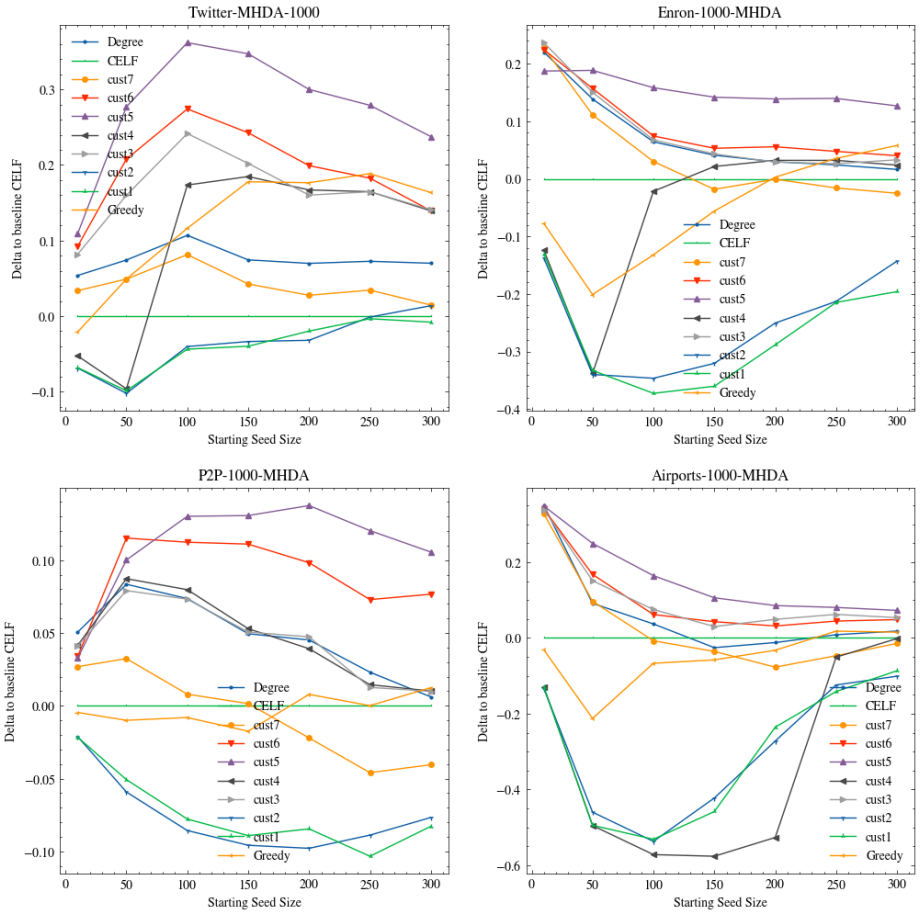


Figure 4: Baseline Performance overall all models on all networks

6.6.1 The data is a curve

Consider a plot of final influence againse seed size as a curve or line. With such a plot there are a few

1. Use the area under the curve. This might seem as the best way of comparing the performance of two methods however it is a very coarse grained approach and will not allow for analysis of performance at different seed size proportions.

2. Compare the heurisitcs at a selection of certain points. This is the approach we will use for the analysis, we may want to find a heuristic for a low seeded proportion and there may be a significant difference with regards to the performance of different methods.

6.6.2 Generalising to networks in General

Using a linear mixed effects model to determine if a particular model is better than the baseline across all four of our test networks.

6.7 A note on perofrmance

The following is a log plot of the mean runtime of each of the hybrid methods against eachother in miliseconds across eaach of the seed sizes as discusessed. Note that this is the total time to compute the seed and then perform a cascade. There may be a small variance based on performance however the network is 1000 nodes so very unlikely to be such a vast diffrence.

Greedy is over 110 times slower than CELF for 300 seed sizes. CELF is Tens of times slower than that of the betweness and clustering heurisitcs.

Clustering and Betweeness are multiple times slower than the basic degree based heurisitics.

In numerical terms on a 64 core zen 5 server cpu on the compute cluster running each trial in a separate process running all of the hybrid heurisitcs took around 23 minutes, CELF took around 35 minutes whilst Greedy ballooned the runtime to 22 hours.

6.8 Tools and Libraries

For the data visualistaion we used the library Matplotlib [9] along with Scienceplots [3] for creating consistent formatting and fonts. The following libraries were used in the code and statistical analysis: Numpy [8], Pandas [13], Scipy [15], Networkx [7]

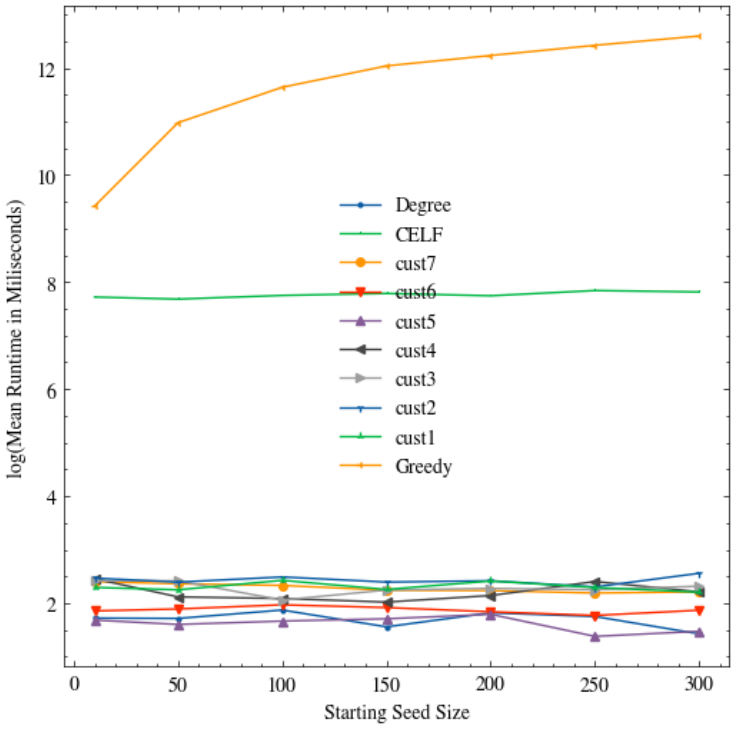


Figure 5: Log Mean Runtime in miliseconds of each method

7 Results and Analysis

In this section we discuss the strategies proposed previously, how they perform across our range of networks, was this behaviour as expected and were there any outliers. We will also analyse each of the methods to test for statistical significance compared to known methods.

7.1 Hybrid Algorithms

7.1.1 Parameters

1. Networks: Each of Twitter, Enron, P2P, Airports networks sampled thirty (30) times using MHDA. Stored and saved to have the exact same networks for use with the 3 models.

2. Baseline Comparisons: Using greedy, CELF and degree as base heuristics to compare against the new hybrid heuristics.

3. Seed Sizes: Each network is made up of one thousand (1000) nodes, when comparing methods the seed sizes to compare will be from 50 to 300 in 50 node incremements with the addition of 10 nodes to represent incredibly small networks.

In this section we discuss the general results across each model on each set of seed sizes. Any standout values are highlighted with comments on their significance or outlier noted.

7.1.2 ICC Analysis

Table 1: Final influence proportion and std. dev of each method on Enron dataset

Method	10	50	100	150	200	250	300
Greedy	0.314 (0.092)	0.389 (0.028)	0.433 (0.029)	0.469 (0.031)	0.529 (0.025)	0.563 (0.030)	0.594 (0.027)
CELF	0.342 (0.039)	0.362 (0.040)	0.370 (0.029)	0.398 (0.029)	0.415 (0.021)	0.450 (0.022)	0.487 (0.017)
Degree	0.331 (0.047)	0.350 (0.037)	0.360 (0.023)	0.381 (0.032)	0.400 (0.020)	0.435 (0.022)	0.464 (0.017)
Custom 1	0.205 (0.160)	0.356 (0.073)	0.416 (0.041)	0.458 (0.042)	0.520 (0.035)	0.560 (0.038)	0.616 (0.025)
Custom 2	0.216 (0.158)	0.355 (0.042)	0.406 (0.069)	0.471 (0.030)	0.543 (0.037)	0.584 (0.033)	0.621 (0.036)
Custom 3	0.346 (0.043)	0.342 (0.034)	0.368 (0.027)	0.379 (0.029)	0.402 (0.026)	0.442 (0.020)	0.464 (0.018)
Custom 4	0.240 (0.155)	0.375 (0.074)	0.415 (0.037)	0.425 (0.025)	0.450 (0.025)	0.472 (0.019)	0.501 (0.020)
Custom 5	0.333 (0.042)	0.359 (0.035)	0.399 (0.038)	0.450 (0.030)	0.485 (0.022)	0.517 (0.031)	0.549 (0.020)
Custom 6	0.332 (0.035)	0.353 (0.033)	0.353 (0.028)	0.386 (0.026)	0.417 (0.024)	0.443 (0.018)	0.476 (0.020)
Custom 7	0.339 (0.041)	0.333 (0.040)	0.356 (0.035)	0.366 (0.037)	0.397 (0.021)	0.426 (0.018)	0.448 (0.018)

On the enron dataset as above in table 1 the standout performers are Custom 2 and Custom 1, these are the two union methods taking a combination of degree and lcc as well as double degree discount and lcc. They have the best overall

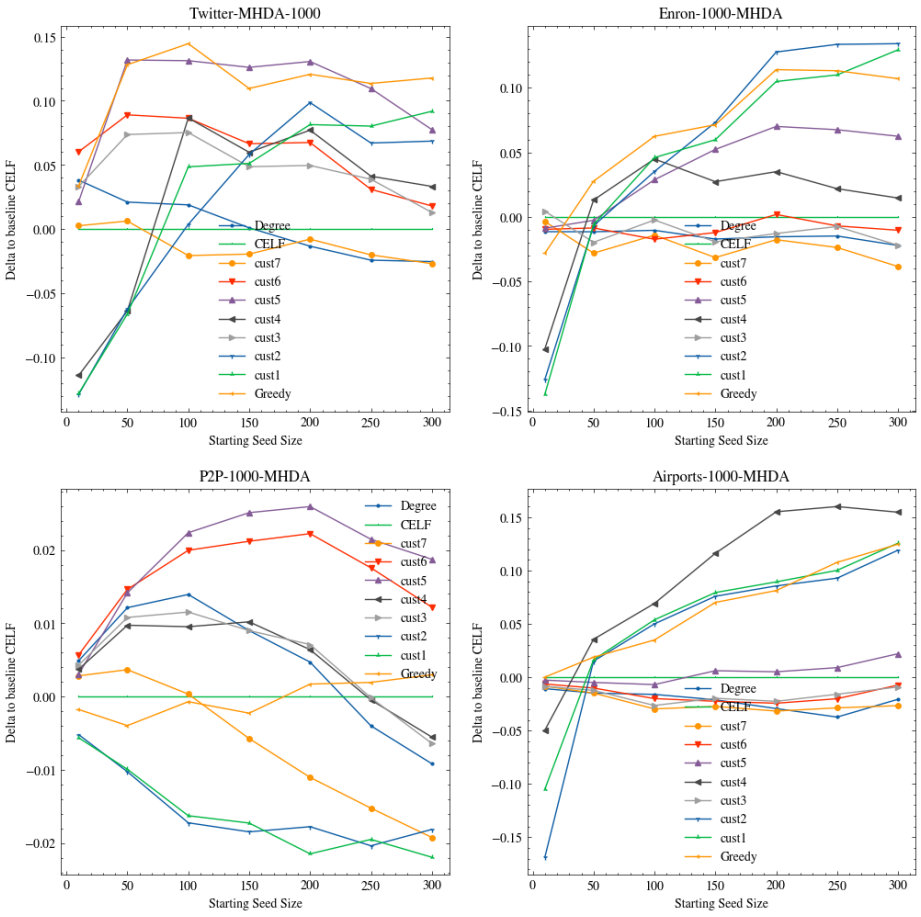


Figure 6: ICC Hybrid Heuristics Performance

performance at seed sizes greater than 100 however are not able to consistently be better than CELF at the lower seed counts.

Custom 5 takes into conisderation both degree and mean neighbour degree, making it incredibly simple to compute and producing a consistently better result than CELF past 100 nodes whilst being minimally worse than CELF at 10 and 50 nodes.

Table 2: Final influence proportion and std. dev of each method on Twitter dataset

Method	10	50	100	150	200	250	300
Greedy	0.188 (0.098)	0.348 (0.069)	0.428 (0.069)	0.450 (0.036)	0.509 (0.049)	0.554 (0.032)	0.605 (0.031)
CELF	0.154 (0.050)	0.220 (0.064)	0.283 (0.054)	0.340 (0.050)	0.388 (0.035)	0.440 (0.041)	0.487 (0.041)
Degree	0.192 (0.044)	0.241 (0.054)	0.302 (0.043)	0.341 (0.032)	0.375 (0.028)	0.416 (0.022)	0.462 (0.025)
Custom 1	0.026 (0.026)	0.154 (0.074)	0.332 (0.121)	0.391 (0.072)	0.470 (0.060)	0.521 (0.054)	0.579 (0.039)
Custom 2	0.025 (0.030)	0.157 (0.073)	0.287 (0.086)	0.398 (0.069)	0.487 (0.057)	0.508 (0.052)	0.556 (0.043)
Custom 3	0.187 (0.038)	0.293 (0.070)	0.358 (0.051)	0.389 (0.032)	0.438 (0.028)	0.479 (0.035)	0.500 (0.027)
Custom 4	0.040 (0.039)	0.156 (0.108)	0.370 (0.055)	0.400 (0.052)	0.465 (0.046)	0.482 (0.035)	0.521 (0.024)
Custom 5	0.176 (0.073)	0.351 (0.053)	0.414 (0.044)	0.466 (0.053)	0.519 (0.046)	0.550 (0.047)	0.565 (0.037)
Custom 6	0.214 (0.047)	0.309 (0.044)	0.370 (0.046)	0.407 (0.036)	0.456 (0.036)	0.471 (0.028)	0.505 (0.026)
Custom 7	0.157 (0.050)	0.226 (0.050)	0.263 (0.057)	0.321 (0.039)	0.380 (0.033)	0.420 (0.025)	0.460 (0.028)

Some slight differences are observed on the twitter dataset, greedy and celf perform very well with custom 5 being the almost equivalent to greedy at all points except 300. Custom 6 along with Custom 3 perform well at the lowe ranges in comaprison to the other heuritics however tail toff towards higher seed counts past 150. There is a very large difference observed at seed sizes of 10 between Custom 2, Custom 1, Custom 4 and the rest of the heuristics, they are almost worthless at sizes 10 and 50 compared to the other methods.

Table 3: Final influence proportion and std. dev of each method on Airports dataset

Method	10	50	100	150	200	250	300
Greedy	0.500 (0.012)	0.524 (0.017)	0.552 (0.013)	0.588 (0.014)	0.609 (0.015)	0.642 (0.011)	0.664 (0.014)
CELF	0.500 (0.015)	0.505 (0.016)	0.517 (0.016)	0.517 (0.016)	0.528 (0.015)	0.534 (0.015)	0.539 (0.014)
Degree	0.489 (0.017)	0.490 (0.018)	0.501 (0.016)	0.496 (0.013)	0.499 (0.016)	0.497 (0.016)	0.519 (0.012)
Custom 1	0.395 (0.193)	0.522 (0.017)	0.571 (0.013)	0.597 (0.017)	0.618 (0.014)	0.634 (0.013)	0.665 (0.009)
Custom 2	0.331 (0.227)	0.520 (0.015)	0.567 (0.015)	0.593 (0.014)	0.614 (0.014)	0.627 (0.014)	0.658 (0.012)
Custom 3	0.492 (0.017)	0.493 (0.020)	0.490 (0.014)	0.498 (0.017)	0.506 (0.016)	0.518 (0.014)	0.530 (0.014)
Custom 4	0.451 (0.147)	0.541 (0.014)	0.586 (0.016)	0.634 (0.018)	0.683 (0.019)	0.694 (0.015)	0.694 (0.016)
Custom 5	0.497 (0.017)	0.500 (0.014)	0.510 (0.013)	0.524 (0.017)	0.533 (0.011)	0.543 (0.013)	0.561 (0.010)
Custom 6	0.494 (0.015)	0.495 (0.017)	0.497 (0.016)	0.495 (0.017)	0.504 (0.012)	0.514 (0.016)	0.532 (0.014)
Custom 7	0.492 (0.014)	0.490 (0.017)	0.487 (0.015)	0.490 (0.017)	0.496 (0.015)	0.505 (0.015)	0.513 (0.015)

The airports dataset is incredibly dense and well connected with a very high mean degree. For staritng seed sizes between 50 and 300 there is no comparison

to custom 4, it provides much better performance than all f the other heuristics. The next best method is Custom 5 which perofrms very closely to CELF from 10 to 100 and surpasses beyond that.

Table 4: Final influence proportion and std. dev of each method on P2p dataset

Method	10	50	100	150	200	250	300
Greedy	0.016 (0.003)	0.072 (0.006)	0.143 (0.009)	0.202 (0.009)	0.266 (0.009)	0.324 (0.009)	0.381 (0.014)
CELF	0.018 (0.005)	0.076 (0.008)	0.143 (0.007)	0.204 (0.011)	0.264 (0.012)	0.322 (0.011)	0.378 (0.010)
Degree	0.023 (0.006)	0.088 (0.009)	0.157 (0.007)	0.213 (0.009)	0.269 (0.010)	0.318 (0.008)	0.369 (0.010)
Custom 1	0.012 (0.002)	0.066 (0.005)	0.127 (0.008)	0.187 (0.008)	0.242 (0.007)	0.303 (0.010)	0.357 (0.008)
Custom 2	0.013 (0.002)	0.066 (0.006)	0.126 (0.008)	0.185 (0.007)	0.246 (0.009)	0.302 (0.009)	0.360 (0.010)
Custom 3	0.022 (0.006)	0.087 (0.005)	0.155 (0.008)	0.213 (0.008)	0.271 (0.009)	0.322 (0.009)	0.372 (0.011)
Custom 4	0.022 (0.006)	0.086 (0.007)	0.153 (0.009)	0.214 (0.009)	0.270 (0.010)	0.322 (0.007)	0.373 (0.009)
Custom 5	0.021 (0.004)	0.090 (0.008)	0.166 (0.010)	0.229 (0.011)	0.290 (0.011)	0.344 (0.010)	0.397 (0.010)
Custom 6	0.024 (0.006)	0.091 (0.007)	0.163 (0.008)	0.225 (0.011)	0.286 (0.008)	0.340 (0.009)	0.391 (0.009)
Custom 7	0.021 (0.005)	0.080 (0.007)	0.144 (0.009)	0.198 (0.008)	0.253 (0.010)	0.307 (0.008)	0.359 (0.008)

On the p2p network the toplogy leads to a more linear and continous set of results compared to the other networks. Custom 5 is a standaout perfoming better than every single other method from 100 and above and being very close to Custom 6 at size 50 whilst being middle of the pack but still better than greedy and celf at size 10.

1. Custom 1: Within the independant cascade model this performs well, closely matching greedy
2. Custom 2: As with Custom 1 this also works well and matches closely to the greedy algorithm
3. Custom 3: Both Perform poorly across the topologies
4. Custom 4: Both perform poorly across the topologies. Outstandign performance on the airports dataset.
5. Custom 5: Perfoming as expected across each of the topologies, airports and enron being an outlier where low seed counts perform poorly but at higher counts performs much better than CELF - for how cheap this is to compute this a really good performer.
6. Custom 6: Performs poorly on all except the p2p - likely due to p2p having a very consistent degree distribution
7. Custom 7: Consistently performs around equal to worse than celf over all topologies.
8. Custom 8:
9. Custom 9: This is a modification of Custom 5 to helpwith the low seed size runs, this does the opposite of the intended effect.

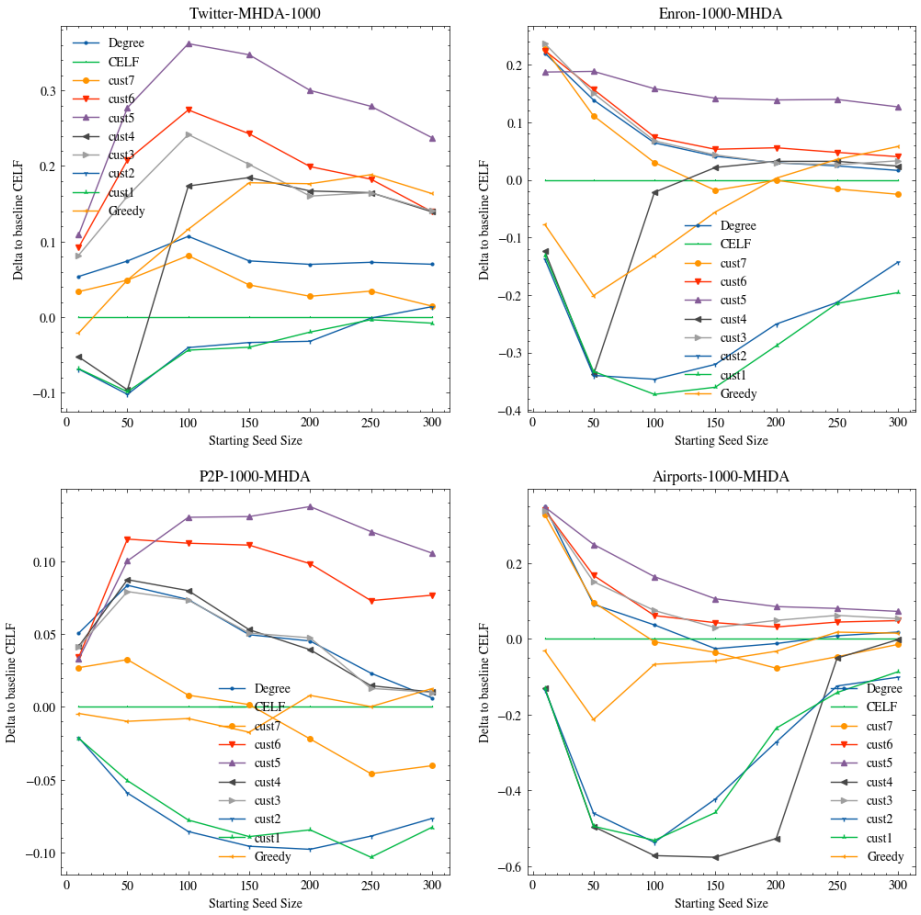


Figure 7: LTM Hybrid Heuristics Performance

7.1.3 LTM Analysis

These are the results from the proposed algorithms in the linear threshold model, each threshold is set at a t distributed uniformily over 0 to 1.

On the enron dataset the most performant heuristic by a large margin is Custom 5 for all seed sizes except 10. Custom 1 and Custom 2 perform very very poorly, similarly to Custom 4 and Greedy.

As with the enron dataset the twitter dataset exhibits a very similar behavirous with Custom 5 being the most performatn by far across the entire range of seed sizes. Similar behaviour for 1 and 2 is shown with really poor final influence spread at low seed sizes.

The airports dataset shows a similar story to the other datasets, Custom 5 is the most perofmant across all seed sizes. The large gap between (x) and (y)

Table 5: Final influence proportion and std. dev of each method on Enron dataset

Method	10	50	100	150	200	250	300
Greedy	0.076 (0.056)	0.246 (0.101)	0.460 (0.102)	0.602 (0.084)	0.702 (0.065)	0.770 (0.042)	0.828 (0.039)
Degree	0.374 (0.070)	0.586 (0.042)	0.656 (0.051)	0.699 (0.031)	0.729 (0.030)	0.758 (0.030)	0.787 (0.029)
cust1	0.023 (0.027)	0.114 (0.048)	0.219 (0.074)	0.298 (0.061)	0.411 (0.078)	0.519 (0.081)	0.574 (0.079)
cust2	0.016 (0.005)	0.107 (0.050)	0.246 (0.091)	0.338 (0.108)	0.448 (0.101)	0.521 (0.100)	0.627 (0.101)
cust3	0.391 (0.078)	0.597 (0.046)	0.660 (0.041)	0.701 (0.029)	0.728 (0.029)	0.760 (0.031)	0.803 (0.023)
cust4	0.030 (0.014)	0.109 (0.085)	0.571 (0.115)	0.680 (0.054)	0.731 (0.032)	0.766 (0.030)	0.794 (0.022)
cust5	0.341 (0.077)	0.636 (0.053)	0.750 (0.027)	0.800 (0.031)	0.838 (0.027)	0.874 (0.020)	0.897 (0.018)
cust6	0.378 (0.088)	0.604 (0.050)	0.667 (0.040)	0.711 (0.032)	0.755 (0.028)	0.782 (0.025)	0.811 (0.020)
cust7	0.381 (0.093)	0.558 (0.044)	0.622 (0.043)	0.640 (0.049)	0.698 (0.038)	0.718 (0.026)	0.745 (0.034)

Table 6: Final influence proportion and std. dev of each method on Twitter dataset

Method	10	50	100	150	200	250	300
Greedy	0.071 (0.035)	0.283 (0.067)	0.418 (0.048)	0.598 (0.054)	0.697 (0.043)	0.768 (0.032)	0.810 (0.032)
Degree	0.146 (0.055)	0.308 (0.085)	0.408 (0.083)	0.494 (0.080)	0.590 (0.077)	0.652 (0.055)	0.716 (0.035)
cust1	0.024 (0.019)	0.135 (0.031)	0.258 (0.051)	0.380 (0.048)	0.501 (0.050)	0.576 (0.053)	0.639 (0.045)
cust2	0.023 (0.013)	0.131 (0.029)	0.261 (0.059)	0.386 (0.051)	0.488 (0.060)	0.578 (0.055)	0.660 (0.041)
cust3	0.174 (0.050)	0.394 (0.078)	0.543 (0.064)	0.622 (0.069)	0.681 (0.053)	0.744 (0.038)	0.787 (0.036)
cust4	0.039 (0.029)	0.137 (0.082)	0.475 (0.069)	0.605 (0.064)	0.687 (0.044)	0.744 (0.054)	0.786 (0.032)
cust5	0.201 (0.063)	0.511 (0.057)	0.663 (0.037)	0.767 (0.030)	0.820 (0.029)	0.858 (0.021)	0.884 (0.016)
cust6	0.184 (0.043)	0.441 (0.070)	0.576 (0.058)	0.663 (0.036)	0.720 (0.045)	0.762 (0.043)	0.786 (0.037)
cust7	0.126 (0.052)	0.283 (0.073)	0.383 (0.075)	0.462 (0.088)	0.548 (0.070)	0.614 (0.051)	0.661 (0.045)

is msot likely to be due to the difference in X for Y because z.

Again Custom 5 is the most performant across all seed sizes above 10 nodes but with a very good performance in 10 being over 8 times better than degree.

1. Custom 1: Consistently poorly performing in comparison to CELF as a baseline.
2. Custom 2: As with Custom 1.
3. Custom 3: Not a good or bad performer. Inconsitent across datasets.
4. Custom 4: Not a good or bad performer. Inconsistent across datasets.
5. Custom 5: Consistently the best perfoming herusitic across all networks from sizes greater than or equal to 50 nodes.
6. Custom 6: Strong performance on P2P especially, Good overall performance above celf for each datasets.
7. Custom 7: Average, not better than CELF for all datasets , strong at low seed counts and very drastically drops off.

Table 7: Final influence proportion and std. dev of each method on Airports dataset

Method	10	50	100	150	200	250	300
Greedy	0.122 (0.114)	0.379 (0.196)	0.650 (0.117)	0.750 (0.108)	0.814 (0.069)	0.881 (0.060)	0.899 (0.044)
Degree	0.503 (0.135)	0.683 (0.043)	0.754 (0.033)	0.783 (0.039)	0.835 (0.027)	0.871 (0.020)	0.902 (0.021)
cust1	0.023 (0.022)	0.097 (0.061)	0.185 (0.068)	0.350 (0.126)	0.611 (0.141)	0.721 (0.092)	0.797 (0.091)
cust2	0.019 (0.014)	0.132 (0.127)	0.180 (0.099)	0.386 (0.166)	0.575 (0.150)	0.739 (0.091)	0.783 (0.092)
cust3	0.492 (0.147)	0.742 (0.047)	0.792 (0.039)	0.838 (0.028)	0.896 (0.016)	0.925 (0.014)	0.937 (0.013)
cust4	0.024 (0.022)	0.096 (0.095)	0.145 (0.059)	0.232 (0.114)	0.320 (0.144)	0.812 (0.043)	0.882 (0.030)
cust5	0.501 (0.172)	0.841 (0.035)	0.881 (0.020)	0.914 (0.014)	0.932 (0.016)	0.943 (0.015)	0.956 (0.011)
cust6	0.494 (0.145)	0.759 (0.050)	0.778 (0.036)	0.851 (0.033)	0.879 (0.025)	0.907 (0.017)	0.932 (0.010)
cust7	0.481 (0.152)	0.688 (0.068)	0.709 (0.041)	0.773 (0.040)	0.770 (0.034)	0.816 (0.039)	0.869 (0.023)

Table 8: Final influence proportion and std. dev of each method on p2p dataset

Method	10	50	100	150	200	250	300
Greedy	0.045 (0.013)	0.194 (0.023)	0.347 (0.042)	0.453 (0.032)	0.564 (0.030)	0.643 (0.022)	0.712 (0.030)
Degree	0.100 (0.033)	0.288 (0.023)	0.428 (0.029)	0.520 (0.023)	0.601 (0.025)	0.666 (0.029)	0.705 (0.019)
cust1	0.028 (0.012)	0.153 (0.022)	0.277 (0.035)	0.381 (0.040)	0.472 (0.035)	0.540 (0.025)	0.616 (0.034)
cust2	0.028 (0.014)	0.145 (0.031)	0.269 (0.024)	0.374 (0.032)	0.458 (0.035)	0.554 (0.029)	0.622 (0.027)
cust3	0.091 (0.027)	0.283 (0.028)	0.428 (0.026)	0.521 (0.029)	0.603 (0.020)	0.656 (0.028)	0.709 (0.022)
cust4	0.091 (0.025)	0.291 (0.036)	0.434 (0.027)	0.523 (0.034)	0.595 (0.022)	0.658 (0.027)	0.709 (0.021)
cust5	0.082 (0.027)	0.304 (0.032)	0.485 (0.033)	0.601 (0.031)	0.694 (0.021)	0.764 (0.025)	0.805 (0.020)
cust6	0.083 (0.025)	0.319 (0.030)	0.467 (0.027)	0.581 (0.030)	0.654 (0.023)	0.716 (0.023)	0.776 (0.022)
cust7	0.076 (0.019)	0.236 (0.032)	0.363 (0.027)	0.472 (0.029)	0.534 (0.025)	0.597 (0.023)	0.659 (0.026)

8. Custom 8: Perfoming same as with custom 1 and 2, very poor.

9. Custom 9: Lower performance than 5 across all datasets and seed sizes.

7.1.4 DCC Analysis

In a similar manner to the linear threshold model custom 5 shows very good behaviour compared to the other heuristics, good initial perofrmance and building a bigger edge the others as seed size increases in the 100-200 range. Other heuristics start to catch up as seed coutns approach 300.

Custom 5 is again the highest performing heuristic over all seed sizes except 10.

Airports shows an interesting behavioru with seed sizes tending past 150, Custom 5 does not follow it's trend compared to in the lower seed proportions. Custom 4 really comes into it's element around 200 nodes and overtakes all of the methods by a large margin.

In the p2p network Custom 5 is again the most performant by a relatively margin compared to the other methods, beating out greedy and celf at 10 and all comparsion methods at 50 and above.

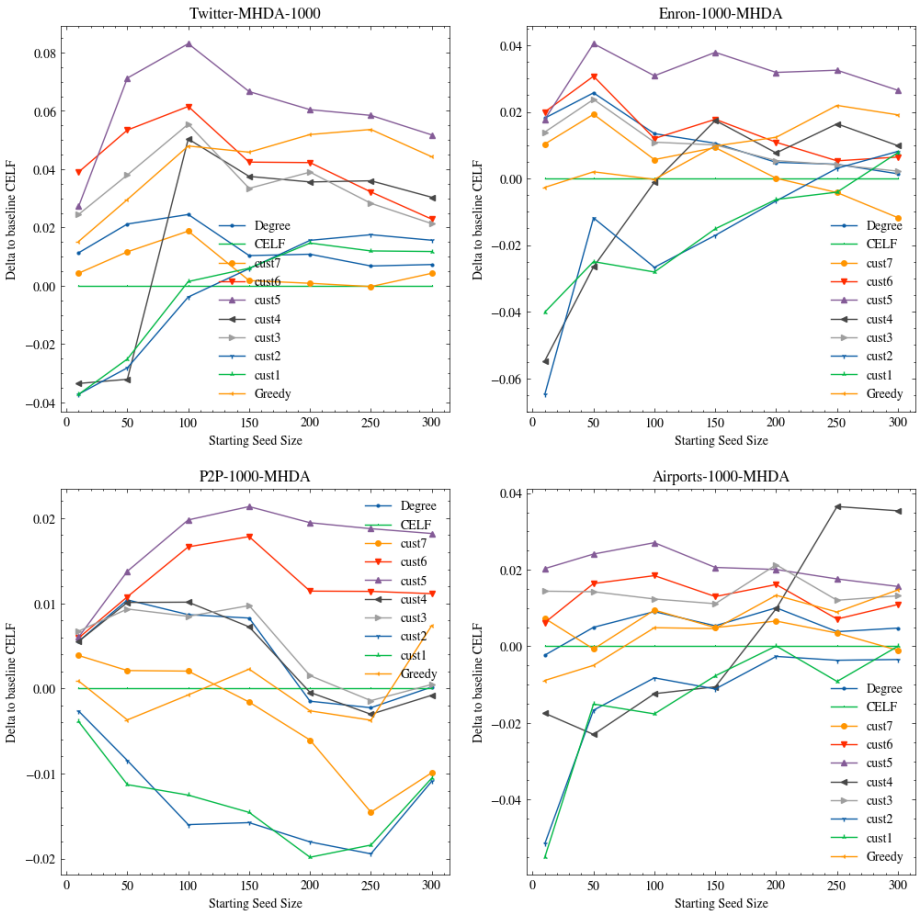


Figure 8: DCC Hybrid Heuristics Performance

Table 9: Final influence proportion and std. dev of each method on Enron dataset

Method	10	50	100	150	200	250	300
Greedy	0.106 (0.020)	0.158 (0.017)	0.220 (0.016)	0.276 (0.016)	0.328 (0.014)	0.381 (0.011)	0.426 (0.014)
CELF	0.109 (0.022)	0.156 (0.018)	0.220 (0.011)	0.266 (0.016)	0.316 (0.013)	0.359 (0.011)	0.407 (0.011)
Degree	0.127 (0.015)	0.182 (0.014)	0.233 (0.013)	0.276 (0.012)	0.321 (0.011)	0.364 (0.011)	0.409 (0.014)
cust1	0.069 (0.042)	0.131 (0.031)	0.192 (0.019)	0.251 (0.019)	0.310 (0.021)	0.355 (0.018)	0.415 (0.015)
cust2	0.044 (0.044)	0.144 (0.022)	0.193 (0.019)	0.249 (0.025)	0.309 (0.016)	0.362 (0.023)	0.415 (0.013)
cust3	0.123 (0.019)	0.180 (0.014)	0.231 (0.014)	0.276 (0.013)	0.321 (0.012)	0.363 (0.011)	0.409 (0.011)
cust4	0.054 (0.043)	0.129 (0.033)	0.219 (0.027)	0.283 (0.012)	0.324 (0.015)	0.376 (0.009)	0.417 (0.012)
cust5	0.126 (0.017)	0.196 (0.013)	0.251 (0.011)	0.304 (0.013)	0.348 (0.013)	0.392 (0.011)	0.434 (0.011)
cust6	0.129 (0.019)	0.187 (0.013)	0.232 (0.012)	0.284 (0.012)	0.327 (0.012)	0.365 (0.012)	0.414 (0.011)
cust7	0.119 (0.019)	0.175 (0.014)	0.226 (0.013)	0.275 (0.016)	0.316 (0.012)	0.355 (0.014)	0.395 (0.011)

Table 10: Final influence proportion and std. dev of each method on Twitter dataset

Method	10	50	100	150	200	250	300
Greedy	0.069 (0.028)	0.148 (0.021)	0.220 (0.021)	0.286 (0.017)	0.343 (0.012)	0.397 (0.016)	0.438 (0.014)
CELF	0.054 (0.014)	0.119 (0.025)	0.172 (0.029)	0.240 (0.019)	0.291 (0.019)	0.344 (0.016)	0.394 (0.017)
Degree	0.065 (0.022)	0.140 (0.027)	0.196 (0.024)	0.250 (0.016)	0.302 (0.015)	0.350 (0.015)	0.401 (0.014)
cust1	0.016 (0.006)	0.093 (0.018)	0.173 (0.018)	0.246 (0.019)	0.306 (0.014)	0.356 (0.017)	0.406 (0.018)
cust2	0.017 (0.008)	0.090 (0.021)	0.168 (0.022)	0.246 (0.019)	0.307 (0.018)	0.361 (0.017)	0.409 (0.012)
cust3	0.078 (0.023)	0.157 (0.023)	0.227 (0.016)	0.273 (0.019)	0.330 (0.017)	0.372 (0.010)	0.415 (0.014)
cust4	0.020 (0.013)	0.086 (0.030)	0.222 (0.029)	0.277 (0.022)	0.327 (0.018)	0.380 (0.016)	0.424 (0.012)
cust5	0.081 (0.022)	0.190 (0.021)	0.255 (0.013)	0.306 (0.013)	0.352 (0.014)	0.402 (0.013)	0.446 (0.013)
cust6	0.093 (0.020)	0.172 (0.015)	0.233 (0.015)	0.282 (0.017)	0.334 (0.017)	0.376 (0.015)	0.417 (0.015)
cust7	0.058 (0.021)	0.130 (0.024)	0.190 (0.027)	0.242 (0.022)	0.292 (0.019)	0.343 (0.014)	0.398 (0.012)

1. Custom 1: Very poor performance, Not a good idea
2. Custom 2: Same as 1, just not a good idea
3. Custom 3:
4. Custom 4:
5. Custom 5: Consistently at towards the top of the chart, always better than celf.
6. Custom 6:
7. Custom 7: A poor performer across all datasets, starts off strong but tails off massively.
8. Custom 8: Tracks similarly to Custom 5 however performs poorly on the p2p netwok
9. Custom 9:

Table 11: Final influence proportion and std. dev of each method on Airports dataset

Method	10	50	100	150	200	250	300
Greedy	0.137 (0.019)	0.192 (0.015)	0.248 (0.018)	0.296 (0.015)	0.342 (0.013)	0.384 (0.012)	0.430 (0.012)
CELF	0.146 (0.018)	0.197 (0.014)	0.243 (0.014)	0.291 (0.015)	0.329 (0.012)	0.375 (0.009)	0.415 (0.012)
Degree	0.144 (0.018)	0.202 (0.022)	0.252 (0.013)	0.297 (0.012)	0.339 (0.009)	0.379 (0.014)	0.420 (0.010)
cust1	0.091 (0.065)	0.182 (0.022)	0.226 (0.020)	0.283 (0.016)	0.329 (0.012)	0.366 (0.012)	0.415 (0.010)
cust2	0.094 (0.061)	0.180 (0.016)	0.235 (0.018)	0.280 (0.016)	0.326 (0.012)	0.372 (0.010)	0.412 (0.009)
cust3	*0.160 (0.019)	*0.211 (0.014)	*0.256 (0.010)	*0.302 (0.011)	*0.350 (0.013)	*0.387 (0.010)	*0.429 (0.011)
cust4	0.129 (0.028)	0.174 (0.025)	0.231 (0.019)	0.281 (0.017)	*0.339 (0.024)	*0.412 (0.012)	*0.451 (0.012)
cust5	*0.166 (0.014)	*0.221 (0.010)	*0.270 (0.011)	*0.312 (0.012)	*0.349 (0.011)	*0.393 (0.012)	*0.431 (0.012)
cust6	*0.152 (0.023)	*0.213 (0.014)	*0.262 (0.013)	*0.304 (0.012)	*0.345 (0.010)	*0.383 (0.012)	*0.426 (0.008)
cust7	*0.153 (0.020)	0.196 (0.018)	0.253 (0.014)	0.296 (0.011)	0.335 (0.011)	0.379 (0.011)	0.414 (0.008)

Table 12: Final influence proportion and std. dev of each method on p2p dataset

Method	10	50	100	150	200	250	300
Greedy	0.017 (0.004)	0.071 (0.006)	0.137 (0.008)	0.201 (0.009)	0.257 (0.011)	0.311 (0.011)	0.370 (0.008)
CELF	0.016 (0.003)	0.075 (0.006)	0.138 (0.009)	0.199 (0.008)	0.259 (0.010)	0.315 (0.011)	0.363 (0.012)
Degree	0.022 (0.005)	0.085 (0.008)	0.147 (0.007)	0.207 (0.008)	0.258 (0.007)	0.313 (0.008)	0.363 (0.007)
cust1	0.012 (0.002)	0.064 (0.006)	0.126 (0.007)	0.184 (0.007)	0.239 (0.007)	0.297 (0.009)	0.353 (0.009)
cust2	0.013 (0.002)	0.066 (0.006)	0.122 (0.006)	0.183 (0.009)	0.241 (0.009)	0.296 (0.007)	0.352 (0.007)
cust3	0.023 (0.005)	0.084 (0.007)	0.147 (0.008)	0.209 (0.010)	0.261 (0.010)	0.314 (0.008)	0.364 (0.008)
cust4	0.022 (0.004)	0.085 (0.007)	0.148 (0.009)	0.206 (0.007)	0.259 (0.009)	0.312 (0.008)	0.362 (0.009)
cust5	0.022 (0.006)	0.089 (0.007)	0.158 (0.008)	0.220 (0.009)	0.279 (0.009)	0.334 (0.009)	0.381 (0.009)
cust6	0.022 (0.005)	0.086 (0.008)	0.155 (0.008)	0.217 (0.008)	0.271 (0.008)	0.326 (0.007)	0.374 (0.009)
cust7	0.020 (0.003)	0.077 (0.006)	0.140 (0.007)	0.197 (0.008)	0.253 (0.009)	0.301 (0.008)	0.353 (0.006)

7.1.5 Subsection of Deg/mnd across parameters

From the above it is clear that it is the best overall cheap heuristics across the 3 models. It generally outperforms CELF on all datasets on all models.

The above analysis as discusessed uses $p = 0.1$ for ICC and DCC with DCC having a 0.7x probability of being turned each time it's attempted call this q. We will test the following parameters against CELF. p in the ranges of 0.05 to 0.25 in 0.05 incremenets. q in the range of 0.5 to 0.9 in 0.1 incremenets, noting that $q = 1$ makes it directly equivalent ot the ICC model.

7.1.6 Methods that Did not work

1. Using just the clustering coefficient of each node can perform either very well on some datasets or is the absolute worst on some others, for this reason it is not included for comparision.

2. Attempting to improve custom 5 (in Custom 9) by means of choosing $logk$ nodes as degree to improve the low seed count performance did not work

and made it perform worse across all areas.

3. Union based approaches
4. Clustering coefficient by itself is overall an incredibly poor performer.
5.

8 Limitations

NOT VERY SMART - LIMITED SCOPE - POOR TIME MANAGEMENT - PROCRASTINATION - MIGHT ACTUALLY DO STUFF WITH DEADLINE IN EXACTLY A MONTH

9 Conclusion

We have shown that some very simple heuristics are able to deliver equal if not better perofrmance than CELF at tiny fractions of the computational cost.

References

[1] James Archbold and Nathan Griffiths. Limiting concept spread in environments with interacting concepts. In *16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017)*, pages 1332–1340. IFAAMAS, 2017.

[2] Henry Franks, Nathan Griffiths, and Sarabjot Singh Anand. Learning agent influence in mas with complex social networks. *Autonomous Agents and Multi-Agent Systems*, 28:836–866, 2014.

[3] John D. Garrett. garrettj403/SciencePlots. September 2021.

[4] Minas Gjoka, Maciej Kurant, Carter Butts, and Athina Markopoulou. Walking in facebook: A case study of unbiased sampling of osns. pages 1 – 9, 04 2010.

[5] Minas Gjoka, Maciej Kurant, Carter T. Butts, and Athina Markopoulou. Practical recommendations on crawling online social networks. *IEEE Journal on Selected Areas in Communications*, 29(9):1872–1892, 2011.

[6] Sanjeev Goyal and Michael Kearns. Competitive contagion in networks, 2012.

[7] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In Gaël Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference*, pages 11 – 15, Pasadena, CA USA, 2008.

[8] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[9] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[10] David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '03, page 137–146, New York, NY, USA, 2003. Association for Computing Machinery.

[11] David Kempe, Jon Kleinberg, and Éva Tardos. Influential nodes in a diffusion model for social networks. In *international colloquium on automata, languages, and programming*, pages 1127–1138. Springer, 2005.

[12] Marcelo N Kuperman. Invited review: Epidemics on social networks. *Papers in physics*, 5(1):0–0, 2013.

[13] The pandas development team. pandas-dev/pandas: Pandas, February 2020.

[14] Vedran Sekara, Arkadiusz Stopczynski, and Sune Lehmann. Fundamental structures of dynamic social networks. *Proceedings of the national academy of sciences*, 113(36):9977–9982, 2016.

[15] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[16] Tianyi Wang, Yang Chen, Zengbin Zhang, Tianyin Xu, Long Jin, Pan Hui, Beixing Deng, and Xing Li. Understanding graph sampling algorithms for social network analysis. *2011 31st International Conference on Distributed Computing Systems Workshops*, pages 123–128, 2011.